

Literature Review and Sequential Baseline

gPPM: Generalized Matrix Operations for
Erasure-Code Encoding and Decoding

PARALLEL COMPUTING PROJECT | MILESTONE 1

Selected paper

S. Li, Q. Cao, S. Wan, W. Xia, and C. Xie
ACM Transactions on Architecture and Code Optimization, 2023

Baseline evaluated: 7 September 2026

Executive Summary

Erasure codes reduce the storage overhead of replication, but encoding and recovery require large finite-field matrix operations. The selected gPPM paper argues that traditional parity-check decoding overlooks independence among failed blocks and may choose an unnecessarily expensive multiplication order. It introduces PPM to partition and parallelize asymmetric codes, then gPPM to select calculation sequences for both asymmetric and symmetric codes [1].

Milestone 1 establishes the reference point required to evaluate those ideas. A scalar, single-threaded SD encoder/decoder was implemented in C, validated over the complete published parameter grid using one representative fault layout per feasible tuple, and benchmarked on exact 32 MiB codewords. All 7,833 representative recovery layouts and all 180 timed trials pass.

Milestone requirement	Evidence in this work
Problem definition	Sections 1 and 2
Literature review with references	Section 2 and References
Target problem metrics	Section 4
Sequential C reference model	Section 3 and repository source
Correctness validation	Section 5.1; sweep_results.csv
Baseline performance results	Section 5.2; benchmark_results.csv

1. Problem Definition

Large storage systems must continue operating when disks, sectors, or nodes become unavailable. Replication is simple but expensive: multiple complete copies consume substantial capacity. Erasure coding instead stores k data units and m parity units so missing data can be reconstructed from the survivors [2]. This reduces redundancy overhead but introduces computation during writes, degraded reads, and repairs.

The computational problem is most visible when parity relationships are represented as a matrix over $\text{GF}(2^w)$. Recovery applies many field multiply-and-XOR operations to full sectors. Modern storage devices can make this coding work a bottleneck rather than a negligible background cost [1,9].

Project question

Can the traditional sequential SD-code matrix decoder be reproduced correctly, measured with hardware-independent and wall-clock metrics, and used as a fair reference for later PPM/OpenMP optimization?

2. Literature Review

2.1 Erasure-code families

Reed-Solomon (RS) codes are the canonical maximum-distance-separable codes: any k of n encoded symbols are sufficient to reconstruct the original data [2]. Cauchy RS, EVENODD, and RDP offer alternative formulations or efficient RAID-oriented implementations [3-5]. In the terminology used by Li et al., these are symmetric-parity codes because parity blocks have comparable dependency structure [1].

Real systems encounter mixed failures. A complete disk may fail while one or more surviving disks contain latent sector errors. Sector-Disk (SD) codes target exactly this model: m disk failures plus s additional sector failures [6]. Locally repairable codes use local and global parity to reduce degraded-read I/O in cloud storage [7,8]. Their unequal parity dependencies make them asymmetric in the gPPM classification.

2.2 Traditional matrix decoding

Parity-check decoding represents a valid stripe B with $H \times B = 0$. Columns for missing blocks form F ; columns for surviving blocks form S . Solving the linear system recovers $B_F = F^{-1} \times S \times B_S$. This formulation is general, but the normal sequence first computes $S \times B_S$ and then applies F^{-1} , treating all missing blocks as one recovery unit [1,6].

2.3 Prior acceleration approaches

Finite-field libraries accelerate the inner multiply-XOR primitive using careful tables and SIMD instructions; GF-Complete and Intel ISA-L are prominent examples [9,10]. GPU and multicore work parallelizes coding at block, equation, or stripe granularity [14]. These techniques improve execution of the chosen operation schedule but do not necessarily reduce the number of full-sector operations. APCM transforms RS parity-check matrices to reduce decoding cost, while EC-Wide focuses on topology and wide-stripe placement [12,13].

2.4 PPM and gPPM

PPM identifies groups of faulty blocks that can be solved independently. It extracts corresponding sub-matrices, schedules independent groups across threads, and solves the dependent remainder after those groups complete [11]. The 2023 journal paper generalizes the idea: gPPM evaluates both normal and matrix-first sequences per matrix using $C = u + c \cdot v$, where u counts nonzero coefficients and v counts coefficients requiring multiplication [1].

The key distinction is complementary optimization. SIMD makes each field operation faster; PPM exposes concurrency and reduces work through partitioning; gPPM also changes operation order. A trustworthy sequential implementation is therefore necessary before speedup claims can be made.

3. Sequential Baseline Design

3.1 SD model

Symbol	Meaning
n	Disks in a stripe
r	Sector rows per disk
m	Complete disk failures tolerated
s	Additional sector failures tolerated
z	Rows containing the s additional failures
w	Field width in $GF(2^w)$
H	Parity-check matrix, $(mr+s) \times (nr)$
$u(M)$	Number of nonzero coefficients in M

The first mr rows of H contain row-local disk-parity equations; the final s rows contain dense sector-parity equations spanning the stripe. This sparse/dense asymmetry is the structure later exploited by PPM. The implementation constructs the FAST matrix with published coefficients and global column exponents.

3.2 Recovery algorithm

1. Construct H from the SD parameters and published coefficients.
2. Extract faulty columns into square matrix F and surviving columns into S .
3. Invert F by Gauss-Jordan elimination over $GF(2^w)$.
4. Compute $T = S \times B_S$, then $B_F = F^{-1} \times T$.
5. Write the reconstructed sectors in place and verify $H \times B = 0$.

$$H \times B = 0 \Rightarrow F \times B_F = S \times B_S \Rightarrow B_F = F^{-1} \times S \times B_S$$

3.3 Software structure

Component	Responsibility
gf.c	$GF(2^{8/16/32})$ arithmetic and counted mult_XORs regions
matrix.c	Dense matrices, nonzero metrics, and inversion
sd_code.c	FAST parity-check matrix, parity and fault layouts
codec.c	H split, normal-sequence encoding/recovery, syndrome
sweep.c	Complete parameter-grid validation
benchmark.c	32 MiB sequential timing harness

4. Metrics and Methodology

4.1 Cost metric

A `mult_XORs` call multiplies a full source region by one GF coefficient and XORs the result into a destination. The counter is independent of processor frequency and sector size, making it suitable for algorithmic comparisons.

$$C = u(S) + u(F^{-1})$$

$$C1 = nr(m+s) + m(mr+s)(z-1) + m^2(r-z)$$

C1 assumes generic nonzero structure in F^{-1} . Particular valid coefficients can cancel to zero, so measured C may be below C1. The strict invariants used here are $C = u(S) + u(F^{-1})$ and $C \leq C1$.

4.2 Correctness method

- Encode random data and require a zero syndrome $H \times B$.
- Save the encoded stripe, erase m disks and s sectors, and recover it.
- Require byte-for-byte equality with the saved stripe and a zero final syndrome.
- Require a nonsingular F , an exact internal operation count, and no C1 over-count.

4.3 Benchmark method

Wall-clock throughput uses `CLOCK_MONOTONIC` around the complete sequential encode/recover call. The primary numerator is useful non-parity data, matching the upstream SD convention. Full-codeword throughput is also recorded. The benchmark validates each result outside the timed region.

5. Results

5.1 Correctness and coverage

Measure	Result
Figure 2 smoke test	PASS; $C=35=C1$
Published coefficient configurations	3,969
Total (configuration, z) points	7,938
Geometrically infeasible points	105
Representative feasible layouts tested	7,833
Round trips passed	7,833 / 7,833
Singular matrices / over-counts / count errors	0 / 0 / 0
Exact C1 / benign under C1	7,649 / 184

These results validate the implementation over the complete parameter grid for the selected deterministic layout at each point. They do not constitute exhaustive enumeration of all possible fault placements.

5.2 Sequential throughput

m	s	Field	C	Encode MiB/s	Decode MiB/s
1	1	GF(2 ⁸)	527	852.60	854.58
1	2	GF(2 ¹⁶)	783	634.68	631.51
1	3	GF(2 ³²)	1039	673.01	671.77
2	1	GF(2 ⁸)	828	444.98	447.81
2	2	GF(2 ¹⁶)	1084	399.48	401.21
2	3	GF(2 ³²)	1340	479.27	476.19

m	s	Field	C	Encode MiB/s	Decode MiB/s
3	1	GF(2 ⁸)	1159	282.56	275.55
3	2	GF(2 ¹⁶)	1415	276.56	279.68
3	3	GF(2 ³²)	1671	352.91	349.51

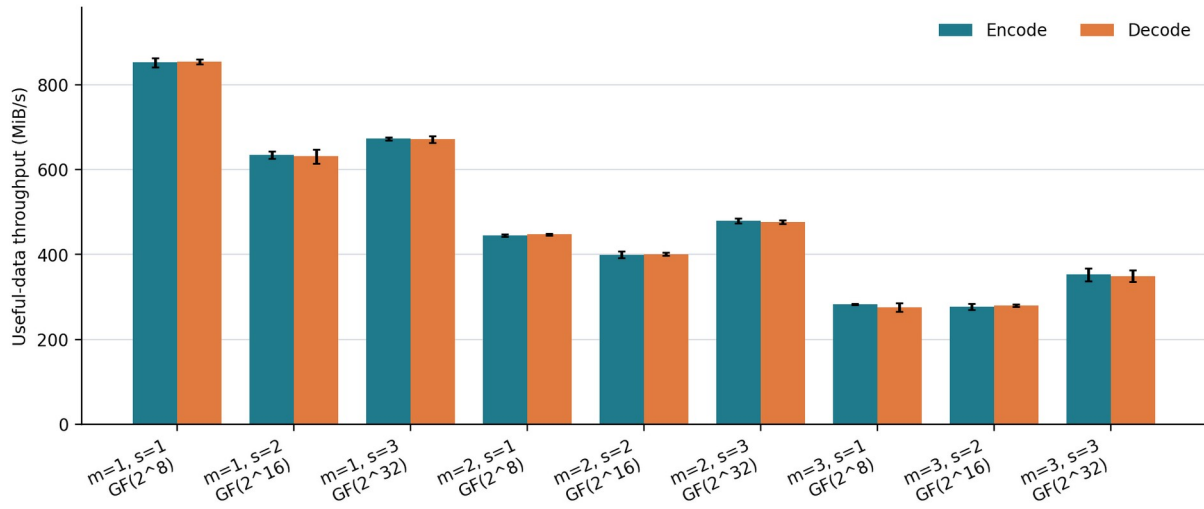


Figure 1. Mean scalar baseline throughput; error bars show one standard deviation across ten trials.

The measured C rises from 527 at (m,s)=(1,1) to 1,671 at (3,3), while decode throughput falls from 854.58 to 349.51 MiB/s. This supports the paper's premise that full-sector operation count is a useful performance indicator. The relationship is not perfectly monotonic because each field width uses a different scalar backend and runtime also includes setup and matrix inversion.

6. Discussion and Limitations

- Internal validity: byte equality, syndrome, operation count, and independent unit tests all pass.
- Coverage: all parameter tuples are tested, but only one deterministic fault placement per feasible tuple.
- Performance scope: results come from one modern CPU and one compiler configuration.
- Comparison scope: the paper used SIMD and different CPUs, so absolute rates should not be compared directly.
- Code scope: the current implementation covers SD only; RS, LRC, PPM, gPPM, and SIMD remain later work.

The baseline is nevertheless appropriate for measuring future speedup because the same executable, datasets, fault layouts, metrics, and validation gates can be retained while only the recovery strategy changes.

7. Conclusion and Milestone Status

Milestone 1's technical requirements are satisfied: the target problem and metrics are defined, relevant literature is reviewed, a sequential C reference model is implemented, correctness is demonstrated over the complete SD parameter grid using representative fault layouts, and 32 MiB baseline throughput is recorded. The next milestone can now implement PPM partitioning and OpenMP while using this baseline as the correctness and performance reference.

Milestone 2 handoff

Preserve the existing validation gates, make operation counters thread-safe, partition H into independent sub-matrices, and compare speedup across thread counts.

References

- [1] S. Li, Q. Cao, S. Wan, W. Xia, and C. Xie, 'gPPM: A Generalized Matrix Operation and Parallel Algorithm to Accelerate the Encoding/Decoding Process of Erasure Codes,' ACM TACO, vol. 20, no. 4, 2023.
- [2] I. Reed and G. Solomon, 'Polynomial Codes over Certain Finite Fields,' Journal of the Society for Industrial and Applied Mathematics, vol. 8, no. 2, 1960.
- [3] J. Blomer et al., 'An XOR-Based Erasure-Resilient Coding Scheme,' ICSI Technical Report TR-95-048, 1995.
- [4] M. Blaum, J. Brady, J. Bruck, and J. Menon, 'EVENODD: An Efficient Scheme for Tolerating Double Disk Failures in RAID Architectures,' IEEE Transactions on Computers, vol. 44, no. 2, 1995.

- [5] P. Corbett et al., 'Row-Diagonal Parity for Double Disk Failure Correction,' FAST, 2004.
- [6] J. S. Plank, M. Blaum, and J. L. Hafner, 'SD Codes: Erasure Codes Designed for How Storage Systems Really Fail,' FAST, 2013.
- [7] C. Huang et al., 'Erasure Coding in Windows Azure Storage,' USENIX ATC, 2012.
- [8] M. Sathiamoorthy et al., 'XORing Elephants: Novel Erasure Codes for Big Data,' Proceedings of the VLDB Endowment, vol. 6, 2013.
- [9] J. S. Plank, K. M. Greenan, and E. L. Miller, 'Screaming Fast Galois Field Arithmetic Using Intel SIMD Instructions,' FAST, 2013.
- [10] Intel Corporation, 'Intel Intelligent Storage Acceleration Library (ISA-L),' 2023.
- [11] S. Li et al., 'PPM: A Partitioned and Parallel Matrix Algorithm to Accelerate Encoding/Decoding Process of Asymmetric Parity Erasure Codes,' ICPP, 2015.
- [12] J. Gu et al., 'Optimizing the Parity Check Matrix for Efficient Decoding of RS-Based Cloud Storage Systems,' IPDPS, 2019.
- [13] Y. Hu et al., 'Exploiting Combined Locality for Wide-Stripe Erasure Coding in Distributed Storage,' FAST, 2021.
- [14] P. Sobe, 'Parallel Reed/Solomon Coding on Multicore Processors,' International Workshop on Storage Network Architecture and Parallel I/Os, 2010.
- [15] Ultra-Low Latency with Samsung Z-NAND SSD.
- [16] Amazon EC2.
- [17] Intel Optane SSD DC P5800X Series.
- [18] Optimizing Software for the Next Gen Intel Optane SSD P5800X.
- [19] H. P. Anvin. 2011. The mathematics of RAID-6.
- [20] Bairavasundaram, Goodson, Pasupathy, Schindler. 2007. An analysis of latent sector errors in disk drives (SIGMETRICS).
- [21] Bairavasundaram, Goodson, Schroeder, Arpaci-Dusseau, Arpaci-Dusseau. 2008. An analysis of data corruption in the storage stack (FAST).
- [22] Blaum, Hafner, Hetzler. 2012. Partial-MDS Codes and their Application to RAID Type Architectures.
- [23] Calder, Wang, Ogun, Nilakantan et al. 2011. Windows Azure Storage (SOSP).
- [24] Ford, Labelle, Popovici, Stokely, Truong, Barroso, Grimes, Quinlan. 2010. Availability in globally distributed storage systems (OSDI).
- [25] Haddock, Bangalore, Curry, Skjellum. 2019. High performance erasure coding for very large stripe sizes (SpringSim).
- [26] Holland, Gibson, Siewiorek. 1993. Fast, on-line failure recovery in redundant disk arrays.
- [27] Hou, Menon, Patt. 1993. Balancing I/O response time and disk rebuild time in a RAID5 disk array.
- [28] Hu, Chen, Lee, Tang. 2012. NCCloud: Applying network coding for storage repair in a cloud-of-clouds (FAST).
- [29] Huang, Xu. 2005. STAR: An efficient coding scheme for correcting triple storage node failures (FAST).
- [30] Iliadis, Haas, Hu, Eleftheriou. 2008. Disk scrubbing versus intra-disk redundancy (SIGMETRICS).
- [31] Jiang, Hu, Zhou, Kanevsky. 2008. Are disks the dominant contributor for storage failures? (FAST).
- [32] Khan, Burns, Plank, Pierce, Huang. 2012. Rethinking erasure codes for cloud file systems (FAST).
- [33] Kim, Seshadri, Dickey, Chiu. 2014. Evaluating phase change memory for enterprise storage systems (FAST).
- [34] Lee, Ipek, Mutlu, Burger. 2009. Architecting phase change memory as a scalable DRAM alternative (ISCA).
- [35] Lee, Lui. 2002. Automatic recovery from disk failure in continuous-media servers (IEEE TPDS).
- [36] Li, Lee. 2014. STAIR codes: A general family of erasure codes for tolerating device and sector failures (FAST).
- [37] Li, Li, Lee, Huang. 2017. Repair pipelining for erasure-coded storage (USENIX ATC).
- [38] Li, Wang, Tian, Wu, Zhang, Xu. 2018. A hierarchical RAID architecture towards fast recovery and high reliability (IEEE TPDS).
- [39] Lin, Gong, Shen, Lee, Shu. 2021. Boosting full-node repair in erasure-coded storage (USENIX ATC).
- [40] MacWilliams, Sloane. 1977. The theory of error-correcting codes, Part I.
- [41] Menon, Mattson. 1992. Distributed sparing in disk arrays (COMPCON).
- [42] Mitra, Panta, Ra, Bagchi. 2016. Partial-Parallel-Repair (PPR) (EuroSys).
- [43] Peterson, Weldon. 1972. Error-Correcting Codes (2nd ed.).
- [44] Pinheiro, Weber, Barroso. 2007. Failure trends in a large disk drive population (FAST).
- [45] Plank. 2008. The RAID-6 liberation codes (FAST).
- [46] Plank. 2013. Open source encoder and decoder for SD erasure codes (Tech Report).
- [47] Plank, Luo, Schuman, Xu, Wilcox-O'Hearn. 2009. A performance evaluation and examination of open-source erasure coding libraries (FAST).
- [48] Qureshi, Srinivasan, Rivers. 2009. Scalable high performance main memory system using phase-change memory technology (ISCA).
- [49] RANDOM.ORG. 2022. Random integer generator.
- [50] Schroeder, Damouras, Gill. 2010. Understanding latent sector errors and how to protect against them (FAST).
- [51] Schroeder, Gibson. 2007. Disk failures in the real world (FAST).
- [52] Tian, Feng, Jiang, Zeng, Chen, Wang, Song. 2007. PRO: A popularity-based multi-threaded reconstruction optimization for RAID (FAST).
- [53] Wan, Wang, Xie, Yang. 2014. S2-RAID: Parallel RAID architecture for fast data recovery (IEEE TPDS).
- [54] Wan, He, Huang, Cao, Li, Xie. 2013. An efficient penalty aware cache for parity-based disk arrays under faulty conditions (IEEE TPDS).

- [55] Wu, Du, Lee, Li, Xu. 2021. StripeMerge: Efficient wide-stripe generation for large-scale erasure-coded storage (ICDCS).
- [56] Yao, Hu, Cheng, Lee, Feng, Wang, Chen. 2022. Optimal data placement for stripe merging in locally repairable codes (INFOCOM).
- [57] Zhong, Li, Wu, Zarkadas, Tao, Mesterhazy, Makris, Yang, Tai, Stutsman, Cidon. 2022. XRP: In-kernel storage functions with eBPF (OSDI).